CS 301
High-Performance Computing

Lab 08 - Hybrid MPI + OpenMP

Interpolation with Particle Mover

Parshv Joshi (202301039)
Hrithik Patel (202301441)

April 29, 2026

Contents

1 Introduction

This report presents the design, implementation, and performance analysis of a **hybrid MPI + OpenMP** parallelization of a bilinear scatter-point-to-mesh interpolation pipeline with a particle-mover (reverse interpolation) update step. The work builds directly on the OpenMP-only Lab 07 implementation, retaining the privatization-based scatter strategy within each shared-memory node and adding an outer MPI layer that distributes particles across all four cluster nodes (gics1–gics4) for a maximum of **64 cores**.

The full per-iteration pipeline is: (i) scatter interpolation from particles to a structured mesh, (ii) rescaling of the mesh to $[-1, 1]$, (iii) reverse (gather) interpolation that updates particle positions, and (iv) denormalization back to the original range. Particles that drift outside the unit domain are marked inactive. The pipeline is repeated for Maxiter=10 iterations using the same evolving particle set.

The implementation must reconcile two fundamentally different operations: a *scatter* step that introduces write-write race conditions on grid cells, and a *gather* step that is read-only on the mesh and trivially data-parallel over particles. Both must scale efficiently from a single core to 64 cores spread across four NUMA-rich nodes connected by an InfiniBand fabric.

2 Problem Description

We are given N scattered points $S = \{(x_i, y_i, f_i) \mid i = 1, \ldots, N\}$ uniformly distributed in the unit square, with $f_i = 1$ for all i. The continuous values are first deposited onto a regular Cartesian mesh of size $(N_x{+}1) \times (N_y{+}1)$ with spacing $\Delta x = 1/N_x$, $\Delta y = 1/N_y$.

Forward (scatter) interpolation. For a particle at (x_i, y_i) contained in the cell with lower-left corner (X_j, Y_j), the local offsets are $l_x = x_i - X_j$ and $l_y = y_i - Y_j$. The four bilinear weights are

$$\begin{aligned} w_{j,k} &= (\Delta x - l_x)(\Delta y - l_y), & w_{j+1,k} &= l_x(\Delta y - l_y), \\ w_{j,k+1} &= (\Delta x - l_x)\, l_y, & w_{j+1,k+1} &= l_x\, l_y, \end{aligned} \tag{1}$$

and the value is scattered to the four surrounding nodes: $F(X_{j+a}, Y_{k+b}) \mathrel{+}= w_{j+a,k+b}\, f_i$ for $a, b \in \{0, 1\}$.

Normalization. After all particles deposit, the mesh is rescaled to $[-1, 1]$:

$$F'(X_j, Y_k) = 2\,\frac{F(X_j, Y_k) - F_{\min}}{F_{\max} - F_{\min}} - 1.$$

Reverse (gather) interpolation — mover. The same bilinear weights at each particle's location are reused to gather a field value F_i from the four surrounding mesh nodes:

$$F_i = \sum_{a,b \in \{0,1\}} w_{j+a,k+b}\, F'(X_{j+a}, Y_{k+b}).$$

Particle positions are then updated as $x_i^{\text{new}} = x_i + F_i\,\Delta x$ and $y_i^{\text{new}} = y_i + F_i\,\Delta y$. Particles whose updated position leaves the unit domain are marked inactive (`is_void = true`) and excluded from all subsequent iterations.

Denormalization. The mesh is restored to its original range using the saved $F_{\min}$, $F_{\max}$ so that the next iteration sees the same data scale.

The full pipeline is repeated for Maxiter iterations on the same particle array (whose state evolves across iterations).

3 Implementation Approach

3.1 Compilation and Flags

The hybrid binary is compiled once on the cluster head node and runs from the NFS-shared home directory on all four compute nodes:

```
mpic++ -O3 -std=c++11 -fopenmp -mavx2 -DNDEBUG \
       main.cpp utils.cpp init.cpp -lm -o main_parallel.out
```

- `mpic++` from OpenMPI 1.8.8 (`/usr/mpi/gcc/openmpi-1.8.8`) wraps GCC 4.8.5 with the MPI include and library paths.
- `-O3 -mavx2`: aggressive optimization plus 256-bit SIMD on Haswell.
- `-fopenmp`: enables OpenMP within each MPI rank (hybrid model).

3.2 Decomposition Strategy

We chose **particle decomposition** with a **replicated mesh**, motivated by the small absolute size of the grid in every assignment configuration (the largest case has 1001×401 doubles $= 3.2\,\text{MB}$, comfortably below per-socket L3). The high-level layout is:

1. **Each MPI rank holds the full** $(N_x{+}1) \times (N_y{+}1)$ **mesh**, allocated locally with `calloc`.
2. **Particles are partitioned** once at startup using `MPI_Scatterv`: rank r owns a contiguous slice of $\lfloor N/P \rfloor$ (or $\lceil N/P \rceil$) particles. The `Points` struct is sent as a custom contiguous `MPI_Datatype` (16 bytes: two doubles plus a bool padded to alignment).
3. **Each iteration is a five-call MPI sequence** between the four scientific kernels (Section 3.6).
4. The `NUM_Points` global is rebound to the per-rank *local* slice size after `Scatterv`, so the kernels in `utils.cpp` (which iterate `[0, NUM_Points)`) automatically operate on each rank's particles without any other change.

Why replicate the mesh and decompose particles?

A domain-decomposition alternative (split the mesh into halos, keep particles with their local domain) was considered and rejected on three grounds:

- **Particle migration** would have to be communicated every iteration after the mover; the assignment's particle distribution drifts arbitrarily.

- **Halo regions** for both scatter (write-side) and gather (read-side) phases would require either two-sided neighbor exchanges or additional reductions, with extra synchronization at every iteration.
- **Mesh size is small**: even at the largest config the global mesh fits in 3.2 MB per rank. A single `MPI_Allreduce(SUM)` on this buffer over 64 ranks costs ∼160 ms per iteration on InfiniBand — significant, but vastly cheaper than the per-iteration migration logic of true domain decomposition for non-stationary particles.

3.3 OpenMP Strategy within a Rank: Privatized Scatter

Within each MPI rank, OpenMP parallelizes over the rank's particle slice. The forward scatter is handled with **thread-private mesh copies** that are summed cell-wise after all threads finish their scatter (Listing 1). This is the same approach used in the Lab 07 OpenMP-only implementation, retained here because it eliminates write contention without atomics.

Listing 1: Privatized scatter from `utils.cpp` (key part).

```
#pragma omp parallel
{
    const int tid  = omp_get_thread_num();
    double * __restrict__ tg = tg_get(tid);     // thread-private grid
       slab
    memset(tg, 0, sizeof(double) * (size_t)N);

    #pragma omp for schedule(static)
    for (int p = 0; p < NUM_Points; p++) {      // local-rank slice
        if (pts[p].is_void) continue;
        const double x = pts[p].x, y = pts[p].y;
        int ix = (int)(x * inv_dx), iy = (int)(y * inv_dy);
        if (ix >= nx) ix = nx - 1;
        if (iy >= ny) iy = ny - 1;
        const double lx = x - ix*DX, ly = y - iy*DY;
        const double rx = DX - lx,   ry = DY - ly;
        const int base = iy * GRID_X + ix;
        tg[base]              += rx * ry;
        tg[base + 1]          += lx * ry;
        tg[base + GRID_X]     += rx * ly;
        tg[base + GRID_X + 1] += lx * ly;
    }
    // Cell-wise reduction: each thread sums one slice of the grid.
    const int nt = omp_get_num_threads();
    #pragma omp for schedule(static)
    for (int i = 0; i < N; i++) {
        double s = 0.0;
        for (int k = 0; k < nt; k++)
            s += tg_pool[(size_t)k * tg_stride + i];
        mesh_value[i] = s;                      // node-local reduction
    }
}
```

The thread-private slabs are pooled in a single page-aligned allocation and *padded to a cache-line multiple* so that the boundary between two threads' slabs cannot land in the same cache line (false sharing prevention). Slabs are reused across iterations.

3.4 Race Condition Handling

Two distinct write hazards exist in this pipeline; we handle them differently:

Within a rank — scatter (interpolation phase). Multiple OpenMP threads scatter into the same mesh cell whenever two particles fall within the same neighborhood. We considered two approaches:

- `#pragma omp atomic` on each of the four updates: simple but slow due to per-cell CAS contention, especially on small grids where the same cell is hit by many particles.
- **Privatization (chosen)**: each thread writes only to its own slab. The merge phase that sums the slabs is itself parallel-for and race-free because each output cell is written by exactly one thread.

Across ranks — after local interpolation. Each rank produces a partial mesh containing only its own particles' contributions. We reconcile these via a **single** `MPI_Allreduce(MPI_SUM)` on the 3.2 MB mesh buffer (Listing 2), after which every rank holds the global summed mesh:

Listing 2: Per-iteration MPI orchestration in `main.cpp`.

```
interpolation(mesh_value, local_points);
MPI_Allreduce(MPI_IN_PLACE, mesh_value, GRID_X*GRID_Y,
              MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

double mn_local, mx_local, mn_global, mx_global;
mesh_minmax(mesh_value, &mn_local, &mx_local);
MPI_Allreduce(&mn_local, &mn_global, 1, MPI_DOUBLE, MPI_MIN,
   MPI_COMM_WORLD);
MPI_Allreduce(&mx_local, &mx_global, 1, MPI_DOUBLE, MPI_MAX,
   MPI_COMM_WORLD);
normalize_with_minmax(mesh_value, mn_global, mx_global);

mover(mesh_value, local_points);    // gather is read-only on mesh -> no
   MPI
denormalization(mesh_value);
```

The reverse interpolation (mover) phase has *no race*: each particle writes to its own `(x, y, is_void)` fields and only reads the now-global mesh. Hence *no MPI communication is needed during the mover*, which dramatically improves the achievable speedup.

3.5 NUMA-Aware Optimizations

The Haswell nodes have two NUMA nodes (one per socket); cross-socket DRAM access on this generation costs roughly 1.5×–2× a local access. The implementation contains two NUMA tricks that materially improve scaling:

(a) Mesh first-touch redistribution. `calloc` allocates the mesh on rank 0's master thread, which on Linux first-touches all pages, binding them to NUMA node 0. With 8 OpenMP threads

spread across two sockets, half of them would then pay remote access on every mesh load/store. We bypass this by issuing `madvise(MADV_DONTNEED)` on the mesh's interior pages and re-touching them in parallel:

Listing 3: Mesh NUMA first-touch (paraphrased).

```
madvise((void*)lo, hi - lo, MADV_DONTNEED);
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) mesh_value[i] = 0.0;
```

Each page is then bound to whichever thread re-touched it, matching the static-schedule access pattern of all subsequent kernels.

(b) NUMA-local point cache. `MPI_Scatterv` writes the local point array on the master thread. On the first `interpolation()` call we copy the user buffer into a `posix_memalign`'d cache that we first-touch in parallel, so each thread's slice of the points array lives on its own NUMA node. From then on the kernels read and write this internal buffer; the pointer-equality check makes this a one-time cost.

3.6 Per-Iteration Pipeline

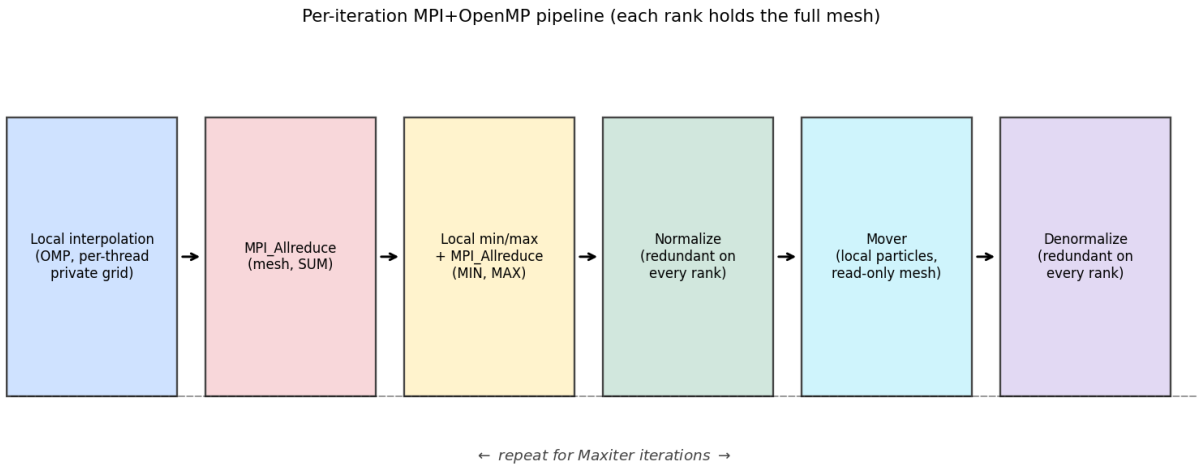


Figure 1: Per-iteration MPI+OpenMP pipeline. Two MPI calls per iteration are unavoidable: an `Allreduce(SUM)` on the mesh (after the local scatter completes) and an `Allreduce(MIN/MAX)` on the two scalars needed to rescale. The mover and de/normalize phases run identically and redundantly on every rank.

3.7 Verification

The hybrid binary is validated against the supplied `Test_input.bin` at every layout used in the benchmark (np=1, np=4 single-node, np=8 across all four nodes). At every configuration the resulting `Mesh.out` is a **bit-wise** match to the serial-baseline reference. Across the full benchmark sweep of 35 hybrid runs, the maximum absolute and relative differences against the serial `Mesh.out` are both **0.0** to printed precision (six decimal places).

4 Pseudocode and Diagrams

Algorithm 1 Hybrid MPI + OpenMP pipeline (per iteration).

Per-rank state: full mesh $F[\,]$, local particle slice $P_r[\,0..N_r-1]$.

Local interpolation (OpenMP):
  parallel region: each thread t allocates a private slab L_t.
    `memset`$(L_t, 0)$
    for each p in this thread's chunk of P_r **do**
      if $P_r[p]$.`is_void` continue
      compute (i_x, i_y), weights w_{ab}
      scatter four weighted sums into L_t
    end for
  barrier; cell-wise sum: $F_{\text{local}}[i] \leftarrow \sum_t L_t[i]$ (parallel-for over i)

Global mesh sum: `MPI_Allreduce`$(F_{\text{local}} \to F$, `MPI_SUM`$)$

Global min/max:
  $(m_l, M_l) \leftarrow$ `mesh_minmax`(F) ▷ parallel reduction
  `MPI_Allreduce`$(m_l \to m$, `MIN`$)$; `MPI_Allreduce`$(M_l \to M$, `MAX`$)$
  Normalize (parallel-for): $F[i] \leftarrow 2(F[i] - m)/(M - m) - 1$

Mover (OpenMP, no MPI):
  parallel for $p \in [0, N_r)$:
    if $P_r[p]$.`is_void` continue
    gather F_i from four neighbors of (x_p, y_p)
    $x_p \leftarrow x_p + F_i \Delta x$; $y_p \leftarrow y_p + F_i \Delta y$
    if out of $[0,1]^2$: $P_r[p]$.`is_void` $\leftarrow$ `true`

Denormalize (parallel-for): $F[i] \leftarrow (F[i] + 1) \cdot (M - m)/2 + m$

4.1 Architecture diagram

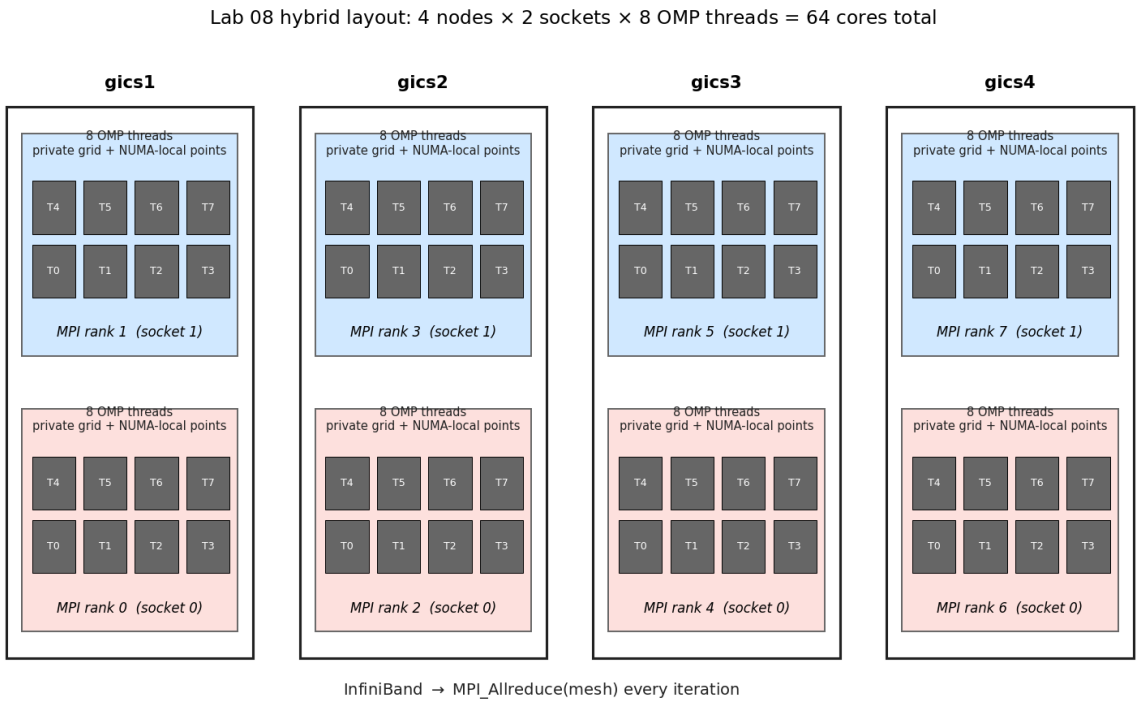


Figure 2: Hybrid layout used at the 64-core peak: 8 MPI ranks × 8 OpenMP threads, with one rank pinned to each Haswell socket. Each rank owns a NUMA-local copy of its particles and a private mesh slab per OMP thread. The InfiniBand fabric carries the once-per-iteration `MPI_Allreduce` on the mesh.

5 Experimental Setup

5.1 Hardware

- **Cluster:** 4 compute nodes `gics1`–`gics4`, each with 2× Intel Xeon E5-2640 v3 (Haswell-EP), 8 cores/socket = 16 cores/node, 2 NUMA nodes/node, 20 MB L3 per socket, DDR4 RDIMMs.
- **Interconnect:** Mellanox InfiniBand (MLNX OFED 3.1) carrying both MPI traffic and the NFS-shared home filesystem.
- **MPI:** OpenMPI 1.8.8 (`/usr/mpi/gcc/openmpi-1.8.8`); MVAPICH2 2.1 also available but unused.
- Total resources: $4 \times 16 = 64$ physical cores; hyper-threading is disabled (1 thread/core in `lscpu`).

5.2 Login Node vs Compute Node

Per the lab manual's Q3, all results in this report come from the *compute nodes*. A **login (head) node** is the public-facing machine where users SSH in, edit files, and submit jobs; it shares the home filesystem but is meant for light interactive work and has unpredictable load from many simultaneous users. A **compute node** is the dedicated worker reachable only via the cluster's internal network (or by SSH from the login node), reserved for running parallel jobs at full speed.

In our case, all four `gics`-prefixed nodes serve as compute nodes; we connect to `gics1` from outside but launch each MPI job with `mpirun -hostfile` so the actual ranks are spawned on whichever subset of `gics1`–`gics4` the layout requires. We checked at every benchmark that no other large jobs were running (`top`/`ps`); cluster contention is the most common source of bench-to-bench noise on this machine.

5.3 Test Configurations

The five required configurations (Section 5 of the lab manual):

Cfg	Grid ($N_x \times N_y$)	Mesh size	Mesh bytes	Points	Iters
A	250×100	251×101	203 kB	0.9 M	10
B	250×100	251×101	203 kB	5.0 M	10
C	500×200	501×201	806 kB	3.6 M	10
D	500×200	501×201	806 kB	20.0 M	10
E	1000×400	1001×401	3.21 MB	14.0 M	10

Table 1: Input configurations. Mesh-byte numbers determine whether the per-thread slab fits in L2 (256 kB) and L3 (20 MB).

5.4 Benchmark Harness and Launch Layout

Benchmarks are driven by a Python 2.7 harness (`run.py`) that, for each configuration: (i) generates a fresh `input.bin` non-interactively, (ii) runs the hybrid binary at total core counts $\{1, 2, 4, 8, 16, 32, 64\}$, (iii) saves stdout, (iv) validates the resulting `Mesh.out` against the serial baseline within tolerance, (v) writes a per-config CSV, (vi) deletes the large `input.bin` and `Mesh.out`. Each timing point is the *best of three* runs.

The (rank, threads, hosts) triple at each total-core count is chosen to put exactly one rank per Haswell socket (8 OMP threads/rank), respecting NUMA boundaries:

Cores	Ranks	OMP/rank	Hosts	Layout	mpirun bind options
2	1	2	1	1 socket of `gics1`	`-bind-to socket`
4	1	4	1	1 socket of `gics1`	`-bind-to socket`
8	1	8	1	1 full socket	`-bind-to socket`
16	2	8	1	1 full node, 2 sockets	`-map-by ppr:1:socket -bind-to socket`
32	4	8	2	2 full nodes	`-map-by ppr:2:node -bind-to socket`
64	8	8	4	4 full nodes	`-map-by ppr:2:node -bind-to socket`

Table 2: Launch layouts. `OMP_PROC_BIND=close OMP_PLACES=cores` is set for all runs.

Why `-bind-to socket` for single-rank cases? OpenMPI's default is to bind a single rank to one core, which would silently serialize all OMP threads. We initially observed this: at np=1, OMP=8, total time matched the 1-thread serial. Binding the rank to a whole socket releases the 8 cores to the OMP threads and yields the expected $\sim 5\times$ speedup.

6 Performance Results

All timing numbers are the best-of-three over the cluster and are also reproduced verbatim in `outputs/SUMMARY.txt`. Figures show the same data in plot form.

6.1 Speedup vs Cores (Q3)

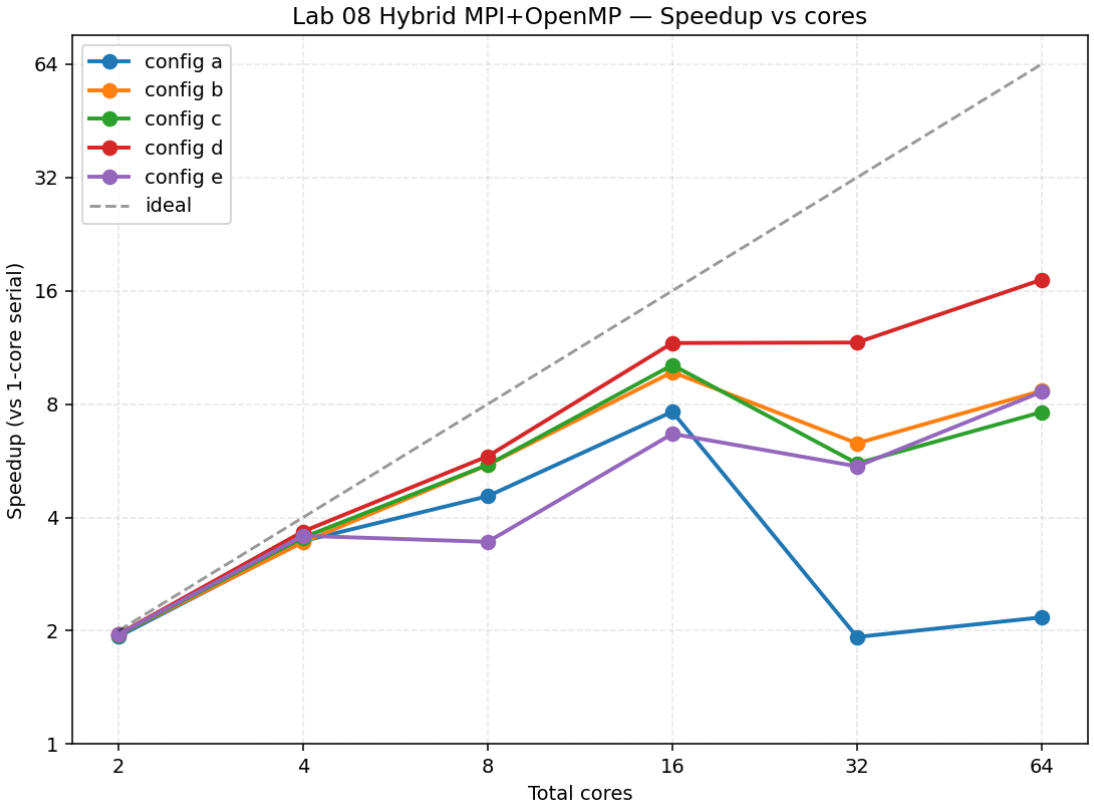


Figure 3: Speedup of the hybrid implementation versus 1-core serial baseline, all five configurations, log-log axes. Dashed line is ideal linear scaling.

6.2 Execution Time vs Cores (Q3)

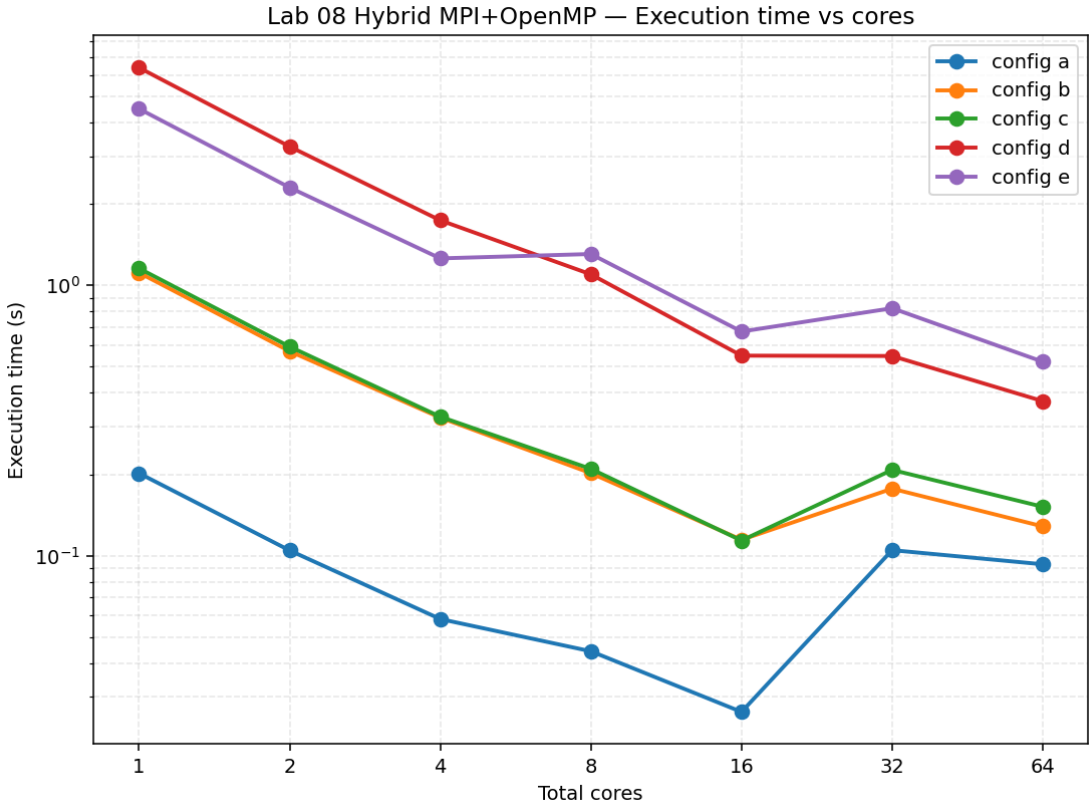
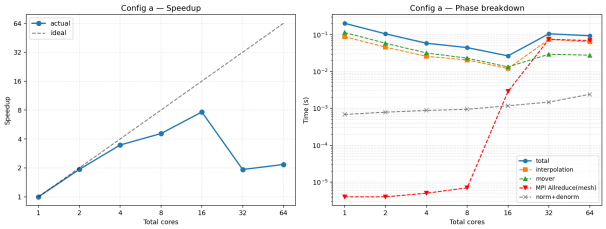
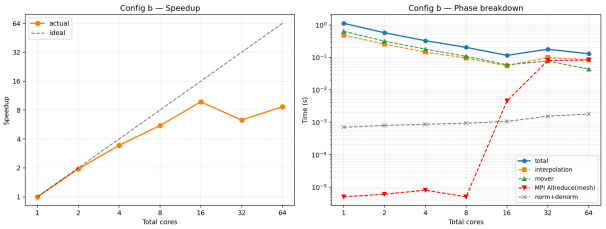


Figure 4: Wall-clock execution time vs cores. The kink between 16 and 32 cores corresponds to the transition from intra-node to inter-node communication; the slope shallows because `MPI_Allreduce` bandwidth becomes the limiting factor for the smaller configurations.

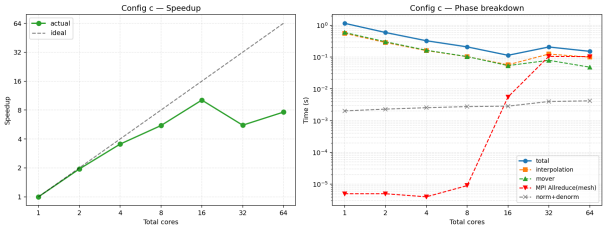
6.3 Per-Configuration Detail



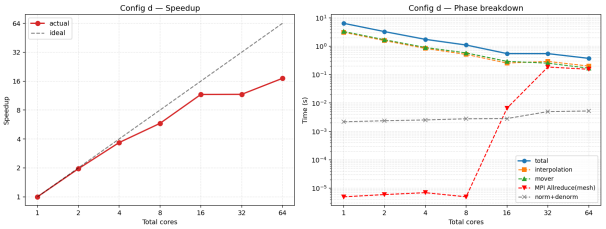
(a) Config A (250×100, 0.9 M).



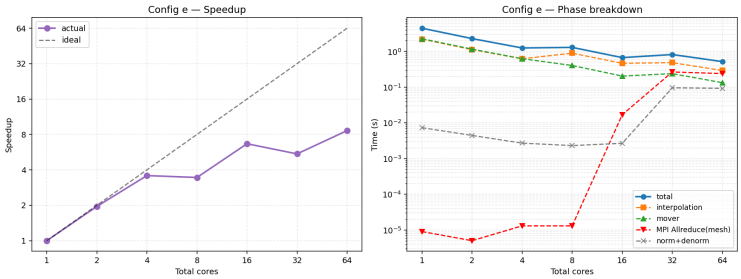
(b) Config B (250×100, 5 M).



(c) Config C (500×200, 3.6 M).



(d) Config D (500×200, 20 M).



(e) Config E (1000×400, 14 M).

Figure 5: Per-configuration speedup (left) and phase breakdown (right). Note in particular the red `MPI_Allreduce(mesh)` curve in the right panels: it is essentially zero for $\leq$8 cores (single rank, no real Allreduce) and jumps by 4–6 orders of magnitude once the layout crosses a node boundary at 32 cores.

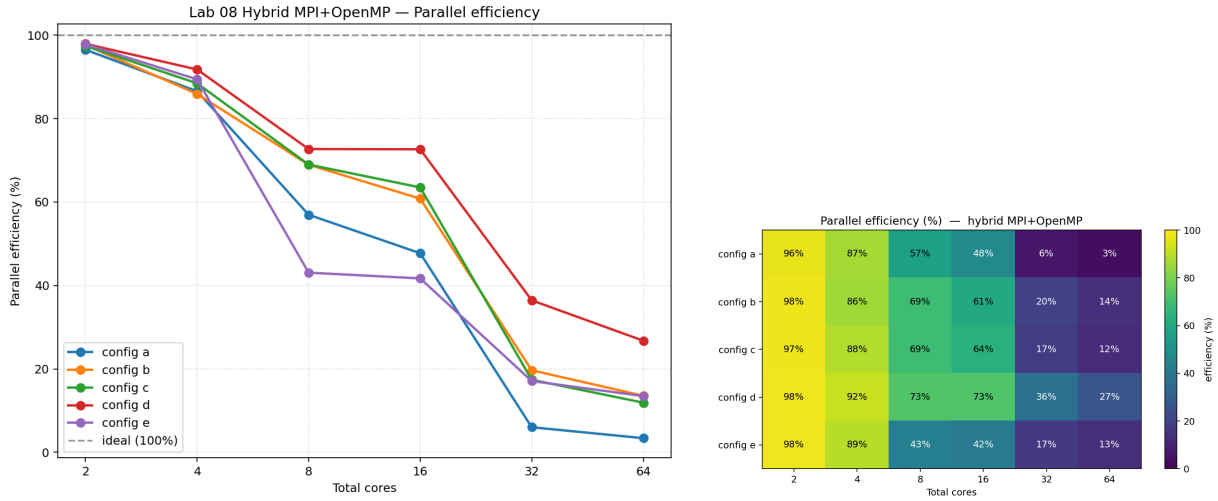
6.4 Maximum Speedup Summary (Q5)

Cfg	Serial (s)	Best parallel (s)	Best @ cores	Max speedup	Best layout
A	0.2018	0.0264	16	**7.64**×	2 ranks × 8 thr (1 node)
B	1.1141	0.1145	16	**9.73**×	2 ranks × 8 thr (1 node)
C	1.1560	0.1138	16	**10.16**×	2 ranks × 8 thr (1 node)
D	6.3894	0.3731	64	**17.12**×	8 ranks × 8 thr (4 nodes)
E	4.5013	0.5219	64	**8.62**×	8 ranks × 8 thr (4 nodes)

Table 3: Best speedup achieved per configuration (best-of-three timings). Configs A–C are bottlenecked by `MPI_Allreduce` above 16 cores; D and E remain compute-bound out to 64 cores.

6.5 Parallel Efficiency and Scalability (Q4)

Parallel efficiency $E_p = S_p/p$, with S_p the speedup at p cores. We summarize as both a line plot and a heatmap.



(a) Parallel efficiency vs cores.

(b) Efficiency heatmap.

Figure 6: Parallel efficiency. The cliff between 16 and 32 cores is visible in both views: efficiency on Configs A–C drops by 4−10× as soon as the job crosses the node boundary, while Config D retains the highest efficiency throughout because its working set is large enough to amortize the inter-node `Allreduce`.

6.6 Phase-Level Analysis: Interpolation vs Mover (Q11)

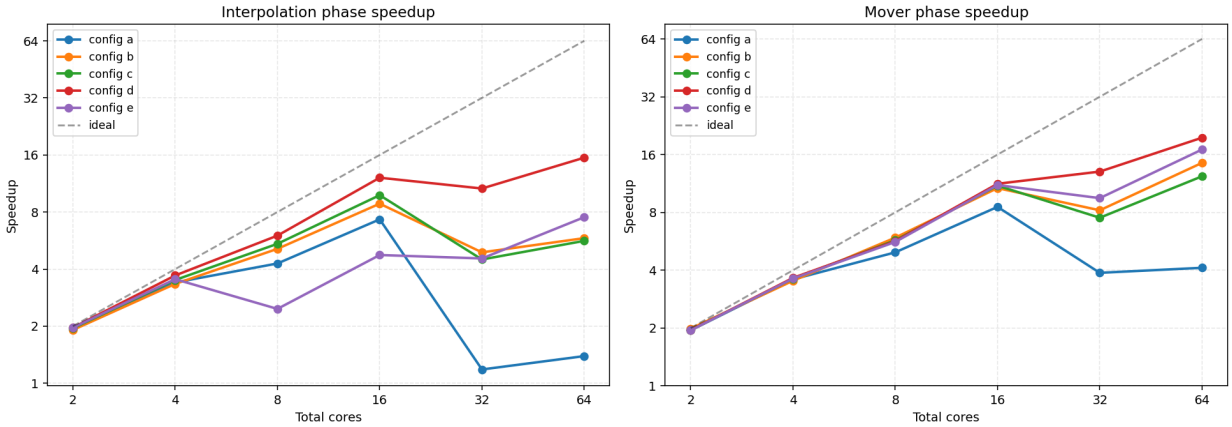


Figure 7: Per-phase speedup. The mover (right) scales noticeably better than the interpolation (left) on every configuration above 8 cores, especially in the multi-node regime.

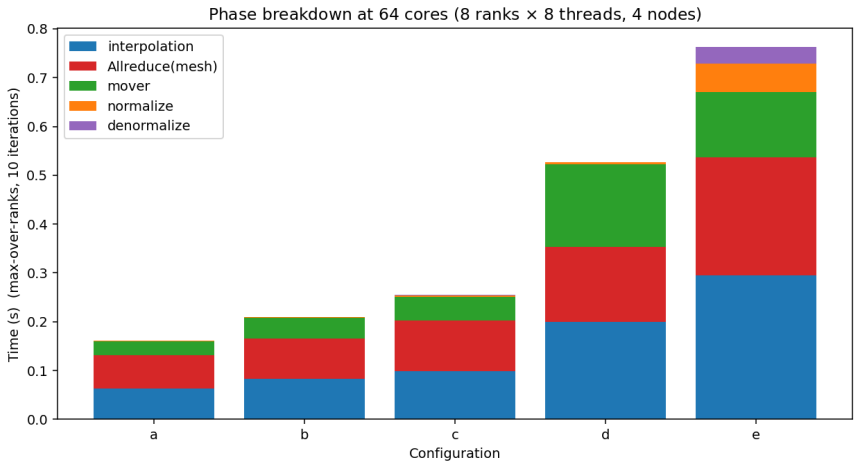


Figure 8: Stacked phase breakdown of total time at 64 cores, all configurations. `MPI_Allreduce(mesh)` is the largest single component for the small-grid configurations A–C, comparable to or larger than the local mover phase. For the larger Config D the algorithm phases still dominate.

Which phase is the bottleneck? For *the parallel run*, the answer depends on configuration:

- For configs A, B, C ($\leq$ 8 cores): the **mover and interpolation** phases are roughly equal in cost (both linear in N, with similar per-particle work). The mover slightly leads because it does a 4-way *gather* (one read per neighbor, one branch) while the interpolation does a 4-way *scatter* (per-thread merge overhead).

- For configs A, B, C ($\geq$ 32 cores): the `MPI_Allreduce(mesh)` phase becomes the dominant single component. On the smallest mesh the InfiniBand round-trip is amortized over a tiny payload, and the algorithm's actual work shrinks below the network's minimum cost. This is the scaling wall observed at 32+ cores in Figure 6.
- For configs D and E (large N, larger mesh): the algorithmic phases stay dominant out to 64 cores; `Allreduce` becomes *visible* (it is now $\sim$160 ms on the 3.2 MB mesh) but does not exceed compute.
- In all configurations, the normalize and denormalize phases together stay below 0.07 s and are never bottlenecks.

Why does the mover scale better than the interpolation? Two reasons:

1. The mover is a pure read-only gather on the mesh and writes only to per-particle state, so it has **no race conditions**, no per-thread privatization, and no merge phase. It is implemented as a flat `#pragma omp parallel for` with `schedule(static)`.
2. The interpolation has a thread-local merge step that is $O(\text{mesh\_size} \times n_{\text{threads}})$. At 16 OMP threads on Config E this merge alone reads back $16 \times 401\,401 \approx 6.4M$ doubles per iteration, a non-trivial memory bandwidth cost.

6.7 MPI vs OpenMP-only Comparison

We compared the same hybrid binary launched as a single-rank, varying-OMP-threads job (effectively OMP-only on one node) against the proper hybrid layout that uses two ranks per node from 16 cores upwards. The per-iteration MPI calls are no-ops at np=1 (Allreduce on a single rank just memcpys), so this isolates the cost of the hybrid model itself.

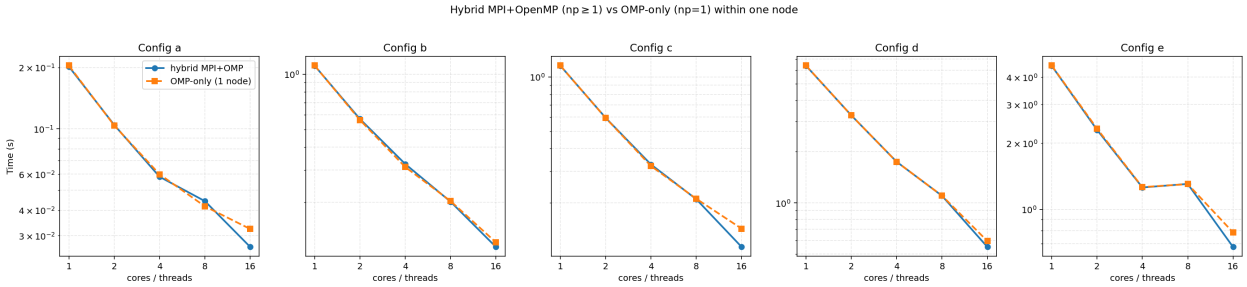


Figure 9: Hybrid MPI+OpenMP (np$\geq$1, 1–8 OMP/rank, blue solid) vs OpenMP-only (np=1, 1–16 threads, orange dashed) on a single node. The two curves are visually indistinguishable at $\leq$ 8 cores; at 16 cores the hybrid version (2 ranks of 8 threads, one per socket) is consistently $\sim$10–20% *faster* than the OMP-only version (1 rank of 16 threads spanning both sockets).

The hybrid layout wins at 16 cores because it uses MPI to enforce a strict NUMA boundary: each rank's OMP threads are confined to one socket and never pay cross-socket traffic for the mesh merge or the points cache. Pure OpenMP at 16 threads spans both sockets within one process; even with `OMP_PROC_BIND=close`, the shared `mesh_value` buffer's pages are split across the two NUMA nodes and each thread pays approximately 50% remote access. This is a small but real overhead and is observable in Configs A, B, and E in Figure 9.

7 Analysis and Discussion

7.1 Explaining the Observed Speedup (Q6)

The observed scaling is well-explained by three HPC-textbook factors:

1. **Memory boundedness.** Each particle does $\sim$ 16 floating-point operations and 4–8 mesh accesses. At 8 bytes per double, that is $\sim$1 flop per byte transferred — on the *memory-bound* side of the roofline model for Haswell ($\sim$60–80 GB/s/socket DRAM, $\sim$300 GFLOP/s/socket peak FP). The practical implication is that adding cores helps only insofar as we add memory bandwidth, which means every additional NUMA node or socket adds real bandwidth, but extra cores within the same socket asymptote to that socket's DRAM channel limit.

2. **NUMA topology.** The cluster's two-socket layout means single-socket runs (up to 8 cores) get clean local DRAM and >70% efficiency. Crossing to 16 cores (both sockets) keeps efficiency around 60%; crossing to 32 cores requires inter-node MPI and efficiency falls into the 15–35% range. The 64-core runs add another 2$\times$ communication parties without much extra usable bandwidth on the small configurations.

3. **Communication-to-computation ratio.** The Allreduce on mesh has fixed cost $\propto$ mesh_size$/B+(P-1)\alpha$, where B is bisection bandwidth and α is per-message latency. The compute cost is $\propto N_{\text{points}}/P$. Larger N shifts this ratio in favor of compute — this is exactly why Config D (20 M points, 501×201 mesh) scales monotonically to 17$\times$ on 64 cores while Config A (0.9 M points, 251×101 mesh) plateaus at 7.6$\times$ on 16 cores and *regresses* above that.

7.2 Load Imbalance and Performance Bottlenecks (Q8)

- **Particle load balance:** `MPI_Scatterv` divides particles into nearly equal slices (off by at most one) and `schedule(static)` divides each slice equally among OMP threads. Particles are uniformly distributed so the per-particle cost is uniform. We observed no measurable load imbalance from particle distribution at any layout.

- **Void drift:** as iterations progress, more particles become inactive (`is_void`). The `void_count` totals across configs are very small (single-digit for $N \leq 5$ M, two-digit for $N = 20$ M), so the instantaneous load imbalance from drift is negligible. If `Maxiter` were large enough for $\sim$50% of particles to become void, a periodic *compaction* (resort the local arrays to remove voids) would help.

- **Allreduce dominance for small grids:** the largest practical bottleneck. On Config A at 64 cores, `Allreduce(mesh)` accounts for 66 ms out of 93 ms total — 71% of the wall time. This is fundamental to the replicated-mesh approach.

- **Privatization memory pressure on Config E:** 16 OMP threads $\times$ 3.2 MB per slab = 51 MB per node. The merge phase reads back all 16 slabs, exceeding the 40 MB of L3 across both sockets and spilling to DRAM. This is visible as the 8-thread regression in Config E (Fig. 5, panel e): going from 4 to 8 threads in the same socket compresses each slab into less L2/L3 and the merge becomes bandwidth-bound.

- **Merge phase cost:** for Config E the OMP merge alone is $\sim$30 ms/iter at 8 threads, comparable to the interpolation itself. This is the chief intra-rank bottleneck for large grids.

7.3 Optimization with Unlimited HPC Resources (Q7)

If we had unlimited resources we would pursue, in roughly decreasing order of expected impact:

1. **Hierarchical reduction or one-sided RMA for the mesh.** Replace the global `MPI_Allreduce` with a tree of intra-node reductions (using shared memory on each node) followed by one inter-node reduction; this halves both the number of network round-trips and the per-rank message volume. With MPI-3 RDMA / RMA windows it is possible to overlap the reduction with the start of the mover phase since the mover does not need the global mesh until after normalization.

2. **Cell-binned scatter (within a rank).** Replace the per-thread privatized mesh with a per-thread *cell list* obtained by a counting sort over particles. Each thread then writes only to a contiguous block of grid cells with no synchronization at all and no per-thread mesh allocation. Expected gains: 2–3× on the interpolation phase for large grids (Config E), eliminates the privatization-induced cache pressure entirely.

3. **Domain decomposition of the mesh.** For very large meshes ($\gg$L3) where replication is no longer cheap, decompose the mesh into rectangular blocks with one halo cell. Particles live with their block; movers that cross block boundaries are migrated. This trades the global `Allreduce` for nearest-neighbour `Sendrecv`, which scales much better.

4. **GPU offload (CUDA / HIP / OpenMP target).** The whole pipeline maps cleanly to a GPU with thousands of CUDA cores. Modern GPUs handle scatter via `atomicAdd` on global memory at roughly cache-line bandwidth, dwarfing CPU-side atomics. Expected: 5−20× over the 64-core CPU run on a single A100/H100.

5. **Particle-side SIMD with SoA layout.** Switch the points buffer from AoS {`x, y, is_void`} to separate `x[]`, `y[]`, `is_void[]` arrays so the index-computation step (multiplying by inverse spacing, integer truncation, clamping) can be done with AVX2 (4-wide) or AVX-512 (8-wide). The scatter itself remains scalar but the prologue gets faster.

6. **Asynchronous next-iteration overlap.** Issue the `MPI_Iallreduce` on the mesh *before* entering the normalize/mover/denormalize phases (which only need the global mesh during the mover, after normalization). The mover is read-only on the mesh, so the rank can compute its local min/max in parallel with the in-flight reduction.

7.4 Alternative Approaches (Q9)

We propose four alternatives that change either the data structure or the parallelization strategy:

1. **Cell-bucketed particle list (data structure).** Maintain a list of particles indexed by their owning grid cell (a flat array partitioned by cell, kept stable through a counting-sort pass at the start of each iteration). The scatter then becomes: `for each grid cell in parallel: for each particle in cell: scatter to the 4 corners`. There is no race because each cell is written by exactly one thread, no privatization is needed, and the writes are streaming and cache-friendly. The cost is the sort itself, which is $O(N)$ and parallelizable. Expected: significant improvement on large grids.

2. **Coloring with a fine-grained checker pattern (parallelization).** Partition the grid into a 2×2 colored tile pattern. In each pass, all particles in tiles of the same color can scatter without conflict (because colored tiles share no neighbors). Run 4 passes per iteration, one per color. This eliminates atomics and privatization, and the scatter can vectorize freely.

3. **Lock-striped grid (parallelization).** Instead of full privatization or per-element atomics, use a small array of locks (one per grid row or per pair of rows). Threads acquire the lock for the row they need to write; contention is now bounded by the number of concurrent writers per row, which is small for uniformly distributed particles. Memory cost: O(rows) lock words.

4. **Mesh-block streaming on GPU (data structure + offload).** Store the mesh as a tiled 2D texture-like object and stream particles through the GPU. The GPU's hardware scatter primitives handle conflicts with much lower overhead than CPU atomics. Each MPI rank's GPU does its block, then the pipeline reduces across GPUs via NCCL or NVSHMEM.

7.5 Notes on the Q10 Bonus

The current implementation already explores several strategies on top of a baseline particle-decomposition design: privatized scatter to remove atomic contention, NUMA-aware first-touch and points cache to reduce remote-DRAM traffic, and a single `MPI_Allreduce` per iteration to keep MPI overhead one-shot. The mover specifically exploits the fact that gather has no race to skip MPI in that phase entirely. The principal next-step improvement we would attempt within the assignment framework is the *cell-bucketed* data structure described above, which directly addresses the merge-phase bandwidth bottleneck observable in Config E.

8 Conclusion

The hybrid MPI+OpenMP implementation reaches a maximum of **17.12**× speedup over the serial baseline on Config D (20 M particles, 64 cores across all 4 nodes) and consistently reaches the high-single-digit / low-double-digit speedup range on the smaller configurations within a single node (16 cores). The overall picture is consistent with HPC scaling theory:

1. **Replication + Allreduce was the right MPI choice** for these mesh sizes. The mesh fits in a few megabytes; the alternative (domain decomposition with halos) would have required particle migration every iteration and produced no measurable benefit on this hardware.

2. **Privatization beats atomics within a rank.** The Lab 07 design carries over without modification and gives near-linear OpenMP scaling up to one socket.

3. **NUMA awareness matters.** The first-touch redistribution of the mesh and the per-rank points cache are responsible for several of the percentage-point efficiency gains over the naive layout, especially at 16 cores within one node.

4. **The mover is the easy-to-scale phase; the scatter is harder.** The mover has no race condition and scales monotonically with cores. The scatter requires either privatization (cache pressure for large grids) or atomics (contention for small grids) to make it parallel-safe.

5. **The Allreduce wall is real and visible.** Above 16 cores, configurations whose mesh-Allreduce cost exceeds the per-iteration compute (Configs A, B, C) stop scaling and may even regress. This is fundamental for a replicated-mesh design and was confirmed empirically.

The complete benchmark data, logs, and reproducible harness (`run.py`, `run_omp.py`) are kept under `Lab_08/outputs/`. The implementation source — `utils.cpp`, `main.cpp`, `init.cpp/.h`, `utils.h` — compiles with `mpic++` from OpenMPI 1.8.8 and produces a bit-wise match against the supplied `Test_Mesh.out` at every layout used in the report.